Campus Landmark A* Route Planner — Complete Implementation Plan

I read your uploaded PRD, Campus Landmark A Route Planner v1.1.0*. It is marked as approved and ready for implementation. The project focuses on finding optimal campus routes using a custom A* algorithm, comparing it with Dijkstra's algorithm, and displaying the results on interactive maps.

Campus_Landmark_AStar_Route_Planner_PRD.md

My recommendation is to keep the original A routing requirements as the core project and add a few practical features that make it more useful, reliable, and impressive as a college mini-project and portfolio project.*

1. What your current PRD already includes

Route calculation

OSMnx map data, walking/driving network modes, landmark snapping, a custom A* search using `heapq`, and Haversine distance.

Algorithm benchmarking

A* versus Dijkstra, path distance, execution time, expanded nodes, and maximum priority-queue size.

Visualization

Folium interactive HTML maps, Matplotlib route diagrams, start/end markers, and console route summaries.

Reliability and testing

Local GraphML caching, one-way edges, disconnected-route detection, coordinate snapping warnings, and five acceptance tests.

These are already a strong foundation. The main gap is that the PRD describes a routing engine and map output, but does not fully define a user-friendly application, its data-management workflow, or how the algorithm's correctness will be demonstrated.

2. Additional features I recommend

I would add the following features in priority order. They build on your existing PRD rather than replacing its requirements.

Priority guide: P0 = essential for a reliable project, P1 = high-value improvement, P2 = optional advanced feature.

1. Searchable landmark directory

P0

Search for buildings, departments, laboratories, libraries, gates, hostels, and sports facilities. Show landmark names and categories in the start and destination selectors.

Benefit: makes routing practical instead of requiring users to enter coordinates.

2. Route directions and distance breakdown

P0
Display ordered route steps, segment distances, total distance, and the landmarks along the path. Clearly distinguish graph-node steps from actual human-readable walking directions.

3. A vs. Dijkstra analytics dashboard*

P0
Show distance equality, runtime, expanded nodes, maximum queue size, and percentage reduction in node expansions. Add comparison charts and a benchmark table.

4. Restricted-path and accessibility controls

P0
Support closed paths, temporary detours, and optional avoidance of stairs where the source data supports it. Provide a way to report incorrect map data.

5. Offline cached routing

P0
Reuse saved GraphML graphs when the internet is unavailable. Display the cache timestamp and show a clear error if no usable graph has been saved.

6. Recent routes and favourites

P1
Save frequently used landmark pairs and recent searches locally. Include a clear-history option.

7. Exportable reports

P1
Export route details and benchmark results to CSV or JSON, and save the interactive Folium map as HTML. Add a PDF report if time permits.

8. Algorithm learning mode

P1

Visualize visited nodes, explored edges, frontier growth, and the \(g(n)\), \(h(n)\), and \(f(n)\) values. Include a step-by-step mode for demonstrations.

9. Estimated travel time

P2
Estimate walking time from route distance and a configurable walking speed. Label it as an estimate, not live traffic or live pedestrian data.

Important technical correction

Your PRD specifies a strongly connected component check before searching. For a directed road graph, being in the same strongly connected component is not necessary for a valid one-way route.

For example, if a one-way path exists from the Main Gate to the Library but no reverse path exists, the route from the gate to the library is still valid.

The safer implementation is:

1. Check whether the source and destination are in the same weakly connected component. If not, a route is impossible.
2. If they are in the same weak component, run the directed A* search normally. The destination might still be unreachable because of one-way restrictions.
3. Optionally use a directed reachability index or cached reachability checks for faster rejection of impossible queries.

This preserves valid one-way routes while still handling disconnected areas efficiently.

A second important correction: the 40% node-expansion reduction should be treated as a benchmark target on a defined set of routes, not as a guarantee for every route. A* is not guaranteed to expand 40% fewer nodes in every graph.

3. Recommended technology stack

Keep the core in Python so that the project stays focused on the AI search algorithms required by your PRD.

Campus_Landmark_AStar_Route_Planner_PRD.md

Component	Recommended technology	Purpose
Programming language	Python 3.10+	Main application and algorithms
Map data	OpenStreetMap + OSMnx	Road and pedestrian networks
Graph processing	NetworkX	Graph conversion, connectivity and components
A* and Dijkstra	Python `heapq`	Custom priority-queue algorithms
Distance heuristic	Haversine formula	Estimate distance to destination

Spatial lookup	SciPy KD-tree or OSMnx nearest-node utilities	Coordinate snapping
Interactive maps	Folium	Route lines, markers and map interaction
Static visualization	Matplotlib	Graph and algorithm comparison plots
Optional web dashboard	Streamlit	Search forms, metrics, charts and result pages
Data formats	GraphML, JSON, CSV	Cached networks, landmarks and exports
Testing	pytest	Unit, integration and performance tests
Code quality	Ruff or Flake8	Style checks and basic linting

Recommended delivery approach: build and test the routing engine first, then add a Streamlit dashboard. Keep Folium as the interactive map renderer and retain the ability to run the project from the command line.

4. Complete system architecture

User interface

Landmark search · Map clicks · Route options · Benchmark dashboard

Application / routing service

Input validation · Landmark resolver · Route request handling

Graph pipeline

OSMnx download · GraphML cache · Edge simplification

Algorithm engine

A* · Dijkstra · Haversine · Search metrics

Validation and reporting

Path verification · Benchmark comparison · Error handling · Export

Results

Optimal path · Interactive map · Directions · Performance report

How a route request should work

1. The user chooses a start landmark and a destination.
2. The application validates both selections.

3. The graph manager loads a suitable cached graph or fetches fresh OpenStreetMap data.
4. The preprocessing module creates the directed graph used by the algorithms.
5. The landmark resolver snaps coordinates to valid graph nodes and checks snapping distances.
6. Connectivity prechecks eliminate obviously impossible requests.
7. A* computes the route; Dijkstra can run separately for benchmarking.
8. The validator checks path continuity, edge costs, and total distance.
9. The visualization layer draws the route and markers.
10. The results panel displays the distance, path, metrics, and any warnings.

Keep the algorithm independent of the UI. That allows you to use the same routing engine from a terminal, a dashboard, or a future mobile application.

5. Recommended project folder structure

Use this structure from the beginning to avoid putting the entire project into one Python file.

campus-landmark-astar/

`src/`

Core application code

`config.py` — settings and configuration

`landmarks.py` — landmark catalog and search

`graph_manager.py` — download, cache and load graphs

`preprocessing.py` — graph conversion and edge weights

`heuristics.py` — Haversine distance

`connectivity.py` — component and reachability checks

`astar.py` — custom A* implementation

`dijkstra.py` — custom Dijkstra baseline

`metrics.py` — timing and search metrics

`route_validator.py` — path and distance verification

`visualization.py` — Folium and Matplotlib

`exports.py` — CSV and JSON reports

`app/`

Optional dashboard

`dashboard.py` — Streamlit entry point

`components.py` — shared UI components

`data/`

Local data

`landmarks.json` — campus landmark records

`graphs/` — cached GraphML files

`exports/` — generated reports and maps

`tests/`

Automated tests

`test_heuristics.py`

`test_astar.py`

`test_dijkstra.py`

`test_connectivity.py`

`test_graph_pipeline.py`

`test_routes.py`

`requirements.txt` · `README.md` · `.gitignore` · `main.py`

Do not commit large graph caches, virtual environments, API credentials, or generated files unnecessarily. Keep the campus landmark catalog and configuration in version control, and provide instructions for downloading or rebuilding the graph.

6. Detailed implementation phases

Phase 1 — Environment and configuration

Tasks

- Install Python 3.10 or newer.
- Create a virtual environment.
- Install and test the required libraries.
- Create the folder structure and configuration module.
- Add logging, a consistent distance unit (meters), and clear exceptions.

Example setup for Windows PowerShell:

```
mkdir campus-landmark-astar
cd campus-landmark-astar

py -m venv .venv
.\.venv\Scripts\Activate.ps1

python -m pip install osmnx networkx scipy folium matplotlib pytest
```

If the `py` launcher is not available, use the full path to your installed Python executable or repair the Python installation and PATH settings.

Deliverable: a working Python environment with a simple script that imports the dependencies.

Phase 2 — Landmark catalog and graph acquisition

Tasks

1. Create `landmarks.json`.
2. Store each landmark's unique ID, display name, latitude, longitude, category, and optional description.
3. Support a place query or bounding box.
4. Download the appropriate graph using OSMnx.
5. Save the graph as GraphML.
6. Load the cached graph on subsequent runs.
7. Handle download failures and invalid cached files.
8. Record the graph's network mode, bounding area, and retrieval time.

Example landmark record:

```
{
  "landmark_id": "MITS_LIB",
  "name": "Central Library",
  "latitude": 13.6288,
  "longitude": 78.5024,
  "category": "Academic"
}
```

The coordinates above are illustrative PRD data, not independently verified coordinates for a real building.

Deliverable: a reusable graph loader and searchable landmark catalog.

Phase 3 — Graph preprocessing and coordinate snapping

The PRD requires conversion from OSMnx's `MultiDiGraph` into a standard `DiGraph`, retaining the minimum-length parallel edge.

Campus_Landmark_AStar_Route_Planner_PRD.md

Tasks

- Inspect the graph's node and edge attributes.
- Ensure edge weights are valid, finite, nonnegative distances.
- For each directed pair (u,v), retain the minimum-length edge.
- Preserve directionality; do not create reverse edges unless the network data allows them.
- Build a spatial index for efficient coordinate snapping.
- Warn if the nearest suitable node is more than 250 meters from the selected coordinate.
- Retain edge geometry when available so that maps follow the actual roads and paths rather than straight lines between graph nodes.

Important: the minimum-length rule is suitable for your stated distance-minimization objective. If the project later models wheelchair accessibility, stairs, closures, or walking preferences, preprocessing must preserve enough attributes to evaluate those constraints.

Deliverable: a clean directed graph and a tested coordinate-to-node resolver.

Phase 4 — Implement the Haversine heuristic

For two coordinates, calculate the great-circle distance using Earth's mean radius of 6,371,000 meters, as specified in the PRD.

$$ \begin{aligned} a={}&\sin^2\left(\frac{\Delta\phi}{2}\right)\\ &+\cos(\phi_1)\cos(\phi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right)\\[4pt] h={}&2R\arcsin\left(\sqrt{a}\right) \end{aligned} $$

Here, latitude and longitude must be converted to radians before evaluating the trigonometric functions.

Tasks

- Implement `haversine_meters(node_a, node_b)`.
- Clamp the intermediate value a to the interval $[0,1]$ to avoid floating-point domain errors.
- Test identical coordinates, known distances, and distant coordinates.
- Verify that the heuristic never overestimates the shortest graph distance in the test graphs.

Deliverable: a reliable heuristic function with unit tests.

Phase 5 — Build the custom A* engine

The core priority is to implement A* directly using `heapq`, rather than relying on a black-box pathfinding wrapper.

Its evaluation function is:

$$ f(n)=g(n)+h(n) $$

- \(g(n)\): actual accumulated edge distance from the source.
- \(h(n)\): estimated distance from the current node to the destination.
- \(f(n)\): estimated total cost through the current node.

Core implementation logic

```
import heapqfrom itertools import countdef astar(graph, start, goal, heuristic):    if start not in graph or goal not in graph:        raise ValueError("Start or goal node does not exist")    if start == goal:        return {            "path": [start],            "distance_m": 0.0,            "nodes_expanded": 0,            "max_frontier_size": 0,        }    sequence = count()    frontier = [        (heuristic(start, goal), next(sequence), start)    ]    came_from = {}    g_score = {start: 0.0}    closed = set()    nodes_expanded = 0    max_frontier_size = 1    while frontier:        _, _, current = heapq.heappop(frontier)        if current in closed:            continue        if current == goal:            path = [goal]
```

This is a starting point for a simplified `DiGraph`, not the complete production module. Add input validation, timing, logging, and tests before integrating it into the application.

Implementation details to enforce

- The heuristic must be callable with the appropriate node IDs and graph coordinates.
- The priority queue uses a sequence counter to avoid comparing arbitrary node IDs when priorities tie.
- The implementation must handle stale queue entries correctly.
- Reopening a node with a better score is safe here and makes the code more robust to heuristic inconsistencies, but it does not make an inadmissible heuristic optimal.
- The code above counts a node when it is expanded, not when it is merely discovered. Use the same definition in Dijkstra.

Deliverable: a custom A* algorithm that returns a path and consistent search metrics.

Phase 6 — Implement Dijkstra's baseline

Implement Dijkstra in `dijkstra.py` using the same graph representation, edge weights, source, and destination as A*.

For Dijkstra:

\[h(n)=0,\qquad f(n)=g(n) \]

Capture:

- Total path distance.
- Expanded nodes.
- Maximum frontier size.
- Execution time.

Do not implement Dijkstra as a separate graph pipeline or use different weights. Otherwise, the comparison will not be meaningful.

Deliverable: two independent algorithms that can be compared on identical inputs.

Phase 7 — Route validation and benchmarking

Create a separate validation layer instead of assuming that a returned path is correct.

Tasks

1. Confirm that the path begins at the selected source and ends at the target.
2. Confirm that every consecutive pair has a valid directed edge.
3. Recalculate the distance from edge weights.
4. Compare A* and Dijkstra distances within a defined floating-point tolerance.
5. Record both runtimes using `time.perf_counter()`.
6. Calculate node-expansion reduction and speedup only when their denominators are valid.
7. Run tests across short, medium, long, and disconnected routes.

\[\text{Expansion reduction (\%)}= \frac{N_D-N_A}{N_D}\times100 \]

Here, \(N_D\) is the number of Dijkstra nodes expanded and \(N_A\) is the number of A* nodes expanded.

A* and Dijkstra must return equal optimal path costs, but they may return different paths when multiple routes have exactly the same cost.

Deliverable: a benchmark report backed by verified route costs.

Phase 8 — Interactive map and dashboard

Implement `visualization.py` first and add the dashboard after the routing engine works.

Map components

- Green start marker.
- Red destination marker.
- Highlighted route polyline.
- Landmark names and categories.
- Route distance and optional estimated walking time.
- Map bounds appropriate to the campus.
- Error messages for unreachable destinations.

Dashboard components

- Project title and campus selector.
- Start and destination search.
- Walking or driving mode.
- Route calculation button.
- Interactive map panel.
- Route summary cards.
- A* vs. Dijkstra comparison table.

- Node-expansion and execution-time charts.
- Export buttons and algorithm learning mode.

For the first version, use a simple single-page dashboard rather than building a separate frontend and backend. This reduces setup time while keeping the core algorithms independently testable.

Phase 9 — Exports, documentation and packaging

- Save route data as JSON using the documented output schema.
- Export benchmark tables to CSV.
- Save interactive maps as HTML.
- Document installation and execution.
- Add a section explaining A*, Dijkstra, Haversine, graph simplification, and connectivity.
- Include test results, limitations, sample screenshots, and a sample benchmark report.
- Provide a reproducible setup using `requirements.txt`.

Deliverable: a project that another student or evaluator can install, run, test, and understand.

7. Complete implementation timeline

Your original PRD proposes a four-day implementation roadmap.

Campus_Landmark_AStar_Route_Planner_PRD.md
For the expanded version, I recommend a 10-day plan that gives proper time to testing and the dashboard.

Day 1

Project setup

Environment, dependencies, configuration, logging and folder structure.

Day 2

Graph acquisition

OSMnx downloads, GraphML cache, landmark catalog and graph modes.

Day 3

Preprocessing

Directed graph conversion, edge validation, connectivity and snapping.

Day 4

A* algorithm

Haversine heuristic, priority queue, path reconstruction and edge cases.

Day 5

Dijkstra algorithm

Baseline implementation, shared graph costs and independent tests.

Day 6

Validation and benchmarking

Distance comparisons, admissibility checks, runtime and expansion metrics.

Day 7

Map visualization

Folium map, route geometry, markers, route summaries and HTML export.

Day 8

Dashboard

Search interface, route selection, metrics, charts and error messages.

Day 9

Advanced features

Route history, CSV/JSON exports, restrictions and learning mode basics.

Day 10

Final testing and submission

Regression tests, documentation, screenshots, demo scenarios and packaging.

These are development targets, not guaranteed completion times. The biggest uncertainty is obtaining suitable campus pedestrian data and verifying that the selected landmarks connect to the downloaded graph.

8. Testing plan and acceptance criteria

Your PRD already specifies five key test scenarios: standard routing, heuristic admissibility, search-space reduction, disconnected nodes, and identical source/destination.

Campus_Landmark_AStar_Route_Planner_PRD.md
Keep those tests and extend them as follows.

Test	What to verify	Expected result
TC-01: Normal route	A* and Dijkstra on a connected graph	Equal optimal costs
TC-02: Heuristic	Sampled reachable node pairs	No overestimation
TC-03: Efficiency	Defined benchmark route set	Measure expansion reduction; target at least 40% for qualifying routes
TC-04: Disconnected route	Nodes with no possible directed path	Clear error; no crash
TC-05: Same node	Source equals destination	Zero distance and no traversed edges
TC-06: One-way path	Valid forward route without a reverse route	Forward succeeds; reverse may fail
TC-07: Invalid landmark	Missing ID or invalid coordinates	Clear validation error
TC-08: Snap distance	Nearest node beyond 250 m	Warning displayed
TC-09: Cache	Graph already saved locally	Reuse compatible cached graph
TC-10: Map output	Valid calculated path	HTML map and route markers generated
TC-11: Edge weights	Invalid, negative or non-finite weights	Reject invalid data
TC-12: Export	Save and reload a route report	Required fields preserved
TC-13: Restrictions	Closed or excluded edges	Returned route respects restrictions
TC-14: Benchmark repeatability	Repeated runs on the same graph	Costs stable; timing variations recorded

Performance tests

The PRD sets a target of 500 milliseconds for path calculations on graphs of up to 50,000 edges, and under three seconds for graph extraction and route computation in areas up to 15 km².

Campus_Landmark_AStar_Route_Planner_PRD.md

Measure these separately:

- Graph download time.
- Graph loading and preprocessing time.
- Coordinate snapping time.
- A* execution time.
- Dijkstra execution time.
- Map rendering time.

Do not include the time needed to download map data when reporting the algorithm's execution time. Network latency and map rendering are separate from search performance.

The 10 ms disconnected-route target also needs careful definition: it is achievable only for a preloaded graph with suitable precomputed connectivity information and does not include graph download or preprocessing.

9. Important design decisions before coding

These are the decisions I would settle early to avoid expensive rework.

Implementation configuration

Choose your preferences to produce a more precise implementation blueprint.

1. Which campus should the application support first?

MITS, Madanapalle
Any campus selected by the user
Small demo campus graph first

2. What type of application do you want?

Command-line Python application
Python + Streamlit dashboard (recommended)
React frontend + Python API backend

3. Which routing mode should be prioritized?

Walking and pedestrian paths
Driving routes
Both walking and driving

4. How much should the first release include?

Core algorithms, tests and map
Complete project with dashboard and exports
Complete project plus advanced features

5. Include visual step-by-step A* learning mode?

Yes — demonstrate how A* explores the graph
No — focus on route finding and benchmarking
Generate my detailed implementation blueprint

10. Final recommended feature scope

For a strong first release, I would prioritize these features:

- ☑ Custom A* and Dijkstra implementations.
- ☑ OpenStreetMap graph extraction and local caching.

- ☑ Searchable campus landmarks and coordinate snapping.
- ☑ Correct handling of directed edges and unreachable destinations.
- ☑ Verified optimal route costs and benchmark metrics.
- ☑ Interactive Folium map and route summary.
- ☑ Streamlit dashboard.
- ☑ Automated tests and CSV/JSON exports.
- ☑ Clear installation instructions and project documentation.

After that, add route restrictions, favourites, accessibility options, and step-by-step algorithm visualization.

The most important principle: get the graph and algorithms correct first, prove A* returns optimal distances, and only then invest time in advanced UI features. This will make your project both technically defensible for an AI laboratory evaluation and useful as a full-stack portfolio demonstration.